Smart Guided Troubleshooting Engine

Transforming Vague Galaxy Device Complaints into Deeplinked, One-Tap Troubleshooting Plans

---

1. Executive Summary & Problem Overview

When smartphone users experience technical issues, they rarely express them in standard technical terms. Instead, support systems receive informal, imprecise descriptions:

- *"Screen flickers and the battery dies fast"*
- *"My phone got slow after the update"*
- *"Swipe gestures go the wrong way after installing an app"*

Current State

Today, resolving these complaints requires significant human intervention:

1. A customer support agent reads unstructured knowledge-base articles (from internal knowledge repositories like SIIS).
2. The agent interprets the complaint, diagnoses root causes, manually selects relevant troubleshooting steps, and arranges them in order.
3. The customer receives text instructions and must manually navigate complex nested device settings (e.g., `Settings > Display > Navigation bar`).

This manual triage process takes roughly **15 minutes per scenario** across millions of customer interactions globally.

Target State

The goal is to build an automated **Smart Guided Troubleshooting Engine** that transforms unstructured natural-language complaints into clean, validated, machine-actionable troubleshooting plans returned in **under 300 ms** for previously encountered issues.

Each plan must:

- Deconstruct the problem into granular, screen-specific actions.
- Order actions logically (least disruptive troubleshooting steps first; destructive/critical operations last).
- Attach exact, verified in-app Settings deeplinks (`bixby://...`) to make each remediation step actionable in a single tap.

---

2. System Architecture & Pipeline

```
Raw Complaint (+ Optional SIIS Raw Knowledge Text)
   │
   ├──► [0] Query Enrichment
   │       • Normalize colloquial text into a canonical technical query
   │       • Generate semantic cache key & 8–10 distinct paraphrases
   │
   ├──► [1] Phase 1: Structure Extraction (LLM / Parsing)
   │       • Extract Goal, categorized Actions, and discrete UI Steps
   │       • Enforce strict length, phrasing, and zero-leak constraints
   │
   ├──► [2] Phase 2: Deeplink Mapping & Action Ordering
   │       • Match target screens to deeplink catalog entries
   │       • Sequence: non-invasive settings first ──► critical/destructive last
   │
   ├──► [3] Fast-Path Semantic Cache
   │       • Cache Hit (<= 300 ms): Return validated plan without LLM invocation
   │       • Cache Miss: Run pipeline [0]–[2], validate schema, write to cache
   │
   └──► [4] Standardized REST API Service
           • Expose endpoints with operational metadata (latency, hit flag, cost)
```

21988027121789119435
Binny Lakra|binny.l|Samsung Electronics|CEO|Device eXperience|Device eXperience CTO|Samsung Research|SRI-Bangalore|Tech Strategy

Core Pipeline Components

Pipeline Component	Functional Responsibilities	Core Engineering Challenge
0. Query Enrichment	Normalizes colloquial phrasing into a structured technical query. Serves as the semantic cache key and generates diverse query paraphrases.	Preventing cache fragmentation; mapping diverse colloquial phrasing to a unified semantic representation.
1. Structure Extraction	Parses unstructured customer-care reference text into a structured `Goal` object containing atomic steps and action categories.	Eliminating LLM hallucination; strictly adhering to source text; enforcing exact field constraints.
2. Deeplink Mapping & Sequencing	Matches step groups to specific device settings screens using semantic and keyword search across masked catalog URIs.	Preventing parent-menu matching; ensuring correct action hierarchy (safe toggles before system resets).
3. Fast-Path Caching	Serves cached, verified plans with sub-300 ms response times. Handles unseen query paraphrases via semantic matching.	High-throughput semantic retrieval without model inference overhead; deterministic response delivery.
4. REST API Service	Exposes a lightweight, containerized microservice returning validated JSON payloads and runtime execution metrics.	Clean containerization, dependable cold-start handling, and robust schema validation.

---

3. Data Inputs & Starter Assets

The problem environment provides standardized datasets to mirror enterprise customer support conditions:

- `queries.json`: Canonical user queries spanning four primary device domains: *Battery*, *Display*, *Camera*, and *Performance*.
- `siis_responses.json`: Pre-cleaned customer-care reference text containing troubleshooting instructions without web URLs or images.
- `deeplinks.json`: A catalog of ~575 masked in-app Settings deeplinks (`bixby://masked/act/...`). Metadata includes descriptive intent strings (`description`, `message`, `qna_description`), control types, and toggle validation rules.
- `bixby://dummy_positive`: A reserved generic placeholder used exclusively when a step opens a valid Settings screen not currently indexed in the catalog.
- `samples/`: Five complete reference input-output pairs illustrating correct extraction and deeplink resolution.
- `schema.py`: Pydantic data models defining the exact input/output interface contract.

---

4. Contract Specifications & Rule Constraints

4.1 Schema Specifications

Outputs must strictly adhere to the following schema rules:

- `goal`: Exact syntax: `Follow these steps to perform this <Topic> Troubleshooting` (or `<Topic> Configuration`).
- `title`: 2 to 3 words, sentence case, identifying the core issue (e.g., `Swipe navigation settings`, `Battery fast drain`).
- `score`: Floating-point value between `0.0` and `1.0` indicating confidence.
- `actionName`: Title Case. Represents exactly **one** physical screen or feature. Multiple steps occurring on the same screen must be grouped under a single action.
- `description`: Exactly 5 to 7 words, starting with `"It will"`, explaining the concrete benefit in plain language.
- `stepGroups[].steps`: Clear, imperative UI steps. One physical interaction per step. No URLs or external links.
- `category`:
  - `auto`: Standard configuration screens reachable via deeplink.
  - `critical`: Disruptive or irreversible operations (e.g., factory reset, restart, firmware update, safe mode). Must be ordered last.
  - `manual`: Physical interventions (e.g., cleaning ports, replacing hardware, visiting service centers). Cannot carry an actionable deeplink.
- `actionableDeeplink`: Verbatim masked URI copied from the catalog matching the target screen.
- `query_variations`: 8 to 10 distinct paraphrases across varied registers (formal, casual, keyword-only, frustrated, typo-inclusive).

4.2 Non-Negotiable Operational Constraints

1. **Zero URL Leaks**: Absolute prohibition of web URLs (`http`, `https`, `www.`, markdown links). Models often inject generic help URLs from pretraining memory; these must be programmatically scrubbed.
2. **Catalog Integrity**: Deeplinks must be matched from `deeplinks.json` using semantic descriptions. Hallucinating or altering URIs is strictly forbidden.

21988027121789119435
Binny Lakra|binny.l|Samsung Electronics|CEO|Device eXperience|Device eXperience CTO|Samsung Resea

3. **No Hallucinated Steps**: Troubleshooting plans must derive purely from provided reference text. If the reference data contains no viable solution, the engine must return an empty list (`contexts: []`) with fallback metadata (`"fallback": "no_match"`).
4. **Pure JSON Delivery**: API responses must never include markdown wrapping (e.g., `json ...`) or conversational preamble.

---

5. System Interface & API Contract

Endpoints

`POST /v1/troubleshoot`

Processes a customer complaint and returns an actionable plan.

- **Request Body**:

```
{
  "query": "phone swipe gestures wrong direction after app install",
  "siis_response": "<optional raw text context>"
}
```

- If `siis_response` is omitted, the engine performs semantic lookup against pre-warmed cache entries.

`GET /health`

Returns HTTP 200 (`{"status": "ok"}`) when the caching layer, model connections, and vector indexes are fully initialized.

---

6. Technical Evaluation Criteria

The system is evaluated across three primary dimensions:

1. Robustness & Hygiene (Automated Gates)

- **Schema Conformance**: 100% adherence to Pydantic definitions and strict field types.
- **Zero Leakage**: Strict elimination of extraneous web links or fabricated URIs.
- **Deterministic Execution**: Consistent action plans for identical or semantically identical inputs.

2. Information Retrieval & Deeplink Precision

- **Screen Resolution Accuracy**: Mapping user actions to exact target screens rather than high-level parent menus.
- **Semantic Paraphrase Hit Rate**: Achieving >= 80% cache hit rates on diverse, unseen query paraphrases.
- **Plan Hierarchy**: Sequencing steps by disruption level (Settings toggles -> system optimizations -> device reboots).

3. Latency & Resource Efficiency

- **Fast-Path Latency**: P95 latency <= 300 ms for cached scenarios.
- **Cold Path Latency**: P95 latency <= 8 s for novel, end-to-end multi-step generations.
- **Cost Predictability**: Tracking token utilization and inference cost per query.

---

7. Common Technical Pitfalls & Edge Cases

1. **Exact-String Cache Keying**: Relying on raw string keys causes paraphrase queries to miss the cache, driving up latency and API costs. Cache keys must be semantically grounded.
2. **Over-Granular vs. Under-Granular Actions**: Creating a separate action for every single tap fragments the UI, while bundling multiple screens into one action breaks deeplinking. The rule is strictly: *One Action = One Screen*.
3. **URL Injection from Model Pretraining**: LLMs tend to generate strings like `Visit samsung.com/support`. Enforce programmatic regex filters on generated steps.
4. **Matching on Masked URI Strings**: Deeplink identifiers (`bixby://masked/act/...`) are obfuscated tokens. Matching must be performed on descriptive metadata fields (`description`, `message`, `qna_description`), never on the URI string itself.
5. **Prompt-Only Constraints**: Asking an LLM to respect word count constraints in natural language is unreliable. Enforce programmatic validation, trimming, and correction loops in the application layer.

21988027121789119435
Binny Lakra|binny.l|Samsung Electronics|CEO|Device eXperience|Device eXperience CTO|Samsung Research|SRI-Bangalore|Tech Strategy

8. Implementation Roadmap (Milestone-Driven)

```
Phase 1: Foundation & Validation
├── Set up schema validation harness using schema.py
├── Implement programmatic validators (word counts, URL stripping, action categories)
└── Validate baseline parsing against reference samples

Phase 2: Semantic Indexing & Deeplink Mapping
├── Index deeplinks.json via dual retrieval (BM25 keyword + dense embeddings)
├── Build screen-resolution logic to map UI steps to masked URIs
└── Implement action ordering logic (safe/least disruptive first, critical last)

Phase 3: Fast-Path Caching Layer
├── Implement semantic normalization and intent extraction for incoming queries
├── Set up persistent local caching with pre-computed query variations
└── Benchmark response latencies (targeting P95 <= 300 ms on hits)

Phase 4: API Assembly & Verification
├── Expose REST endpoints (/v1/troubleshoot, /health)
├── Implement error boundaries and graceful fallbacks ("no_match", "no_siis_context")
└── Run end-to-end stress tests evaluating cold-start, cache hits, and schema compliance
```

21988027121789119435
Binny Lakra|binny.l|Samsung Electronics|CEO|Device eXperience|Device eXperience CTO|Samsung Research|SRI-Bangalore|Tech Strategy

Appendix A: Data Contract Reference (`schema.py`)

```
from enum import Enum
from typing import Dict, List, Optional
from pydantic import BaseModel

class BaseDeeplink(BaseModel):
    deeplink: str

class Deeplink(BaseDeeplink):
    description: str
    message: Optional[str] = ""
    classes: Optional[Dict[str, str]] = None
    originalType: Optional[str] = None

class Condition(str, Enum):
    greater = "greater"
    equal = "equal"
    less = "less"

class ResultTypes(str, Enum):
    boolean = "boolean"
    intNum = "integer"
    string = "str"
    floatNum = "float"

class actionCategory(str, Enum):
    auto = "auto"
    manual = "manual"
    critical = "critical"

class ValidationDeepLink(BaseDeeplink):
    key: str
    resultType: Optional[ResultTypes] = None
    condition: Optional[Condition] = None
    value: Optional[str] = None

class StepGroup(BaseModel):
    steps: List[str]
    validationDeeplink: Optional[ValidationDeepLink] = None
    actionableDeeplink: Optional[Deeplink] = None

class Action(BaseModel):
    actionName: str
    description: str
    stepGroups: List[StepGroup]
    category: Optional[actionCategory] = actionCategory.manual

class Goal(BaseModel):
    goal: str
    title: str
    actions: List[Action]
    score: float

class ContextDeeplinkResponse(BaseModel):
    """RAG response containing a list of Goal objects."""
    contexts: List[Goal] = []
```

Binny Lakra|binny.l|Samsung Electronics|CEO|Device eXperience|Device eXperience CTO|Samsung Resea

21988027121789119435

Appendix B: Worked End-to-End Example (API 200 / `results.jsonl`)

The following demonstrates a complete, compliant response body returned by `POST /v1/troubleshoot` and serialized per line in `results.jsonl`:

```
{
  "query": "The mobile phone swipe navigation moves up or down instead of left or right after downloading an a
  "query_variations": [
    "Ever since I installed a new app, swiping on my phone scrolls up and down instead of going left or right.
    "Why does my phone swipe vertically when I try to swipe sideways after downloading an app?",
    "phone swipe gestures wrong direction after app install",
    "I downloaded an application yesterday and now the swipe navigation on my Galaxy moves up or down when it
    "Screen navigation gestures are misbehaving after an app download; horizontal swipes register as vertical
    "My phone's gesture navigation got messed up by a new app and swipes go the wrong way.",
    "What should I do when swiping left or right on my phone scrolls the screen up and down instead?",
    "Swipe navigation broken after installing app.",
    "This is so annoying - I can't swipe sideways anymore since installing that app, everything just scrolls v
    "Navigation swipes on my Samsung phone respond in the wrong axis after a recent app installation."
  ],
  "response": {
    "contexts": [
      {
        "goal": "Follow these steps to perform this Swipe Navigation Troubleshooting",
        "title": "Swipe navigation settings",
        "score": 0.93,
        "actions": [
          {
            "actionName": "Configure Navigation Bar Settings",
            "description": "It will let you choose navigation type",
            "category": "auto",
            "stepGroups": [
              {
                "steps": [
                  "Navigate to and open Settings.",
                  "Tap on Display.",
                  "Tap on Navigation bar.",
                  "Select your preferred navigation type between Buttons and Swipe gestures.",
                  "Optionally toggle on Gesture hint to display guidance lines at the bottom of the screen."
                ],
                "actionableDeeplink": {
                  "deeplink": "bixby://dummy_positive",
                  "description": "Open navigation bar settings under Display",
                  "message": "Choose navigation type in Display settings"
                },
                "validationDeeplink": null
              }
            ]
          }
        ]
      }
    ]
  },
  "meta": {
    "latency_ms": 212,
    "cache_hit": true,
    "model": "gpt-4o-mini",
    "cost_usd": 0.0
  }
}
```

21988027121789119435
Binny Lakra|binny.l|Samsung Electronics|CEO|Device eXperience|Device eXperience CTO|Samsung Research|SRI-Bangalore|Tech Strategy

Appendix C: System Performance & Evaluation Template (`metrics.md`)

This markdown report template standardizes system benchmarking, quality scoring, latency percentiles, and ablation analyses across implementations:

21988027121789119435
Binny Lakra|binny.l|Samsung Electronics|CEO|Device eXperience|Device eXperience CTO|Samsung Research|SRI-Bangalore|Tech Strategy

# System Performance Metrics & Evaluation Report
Model(s): <provider/model id>
Embeddings: <embedding model id>
Environment: <vCPU count / RAM / OS spec>

---

1. Schema & Rule Compliance
Evaluated on sample datasets and held-out validation scenarios.

Metric	Target	Measured Value
Schema-valid output lines	>= 99%	%
Rule compliance (Goal / Title / Description syntax)	>= 95%	%
Absolute URL leaks	0	0
Deeplink catalog validity (exact URI match)	100%	%
Auto actions carrying valid actionable deeplink	>= 90%	%

## 2. Accuracy Benchmarks
Evaluated against reference ground truth scenarios across Battery, Display, Camera, and Performance.

Evaluation Metric	Scale / Anchor	Score
Step accuracy (completeness, correctness, ordering)	0.0 - 3.0	
Deeplink relevance (exact target screen vs. parent menu)	0.0 - 2.0	

## 3. Latency Benchmarks (N >= 30 requests per path)

Execution Path	Target (P95)	P50 (ms)	P95 (ms)
Cache hit - exact query match	<= 300 ms		
Cache hit - unseen semantic paraphrase	<= 300 ms		
Cold query - full pipeline extraction & mapping	<= 8000 ms		

## 4. Operational Cost & Cache Efficacy

Metric Item	Target	Measured Value
Cold query average inference cost	Tracked	$0.00
Cache hit inference cost	$0.00	$0.00
Semantic cache hit rate (on unseen paraphrases)	>= 80%	%
Cost derivation method	-	(prompt tokens + completion tokens) x rate

---

## 5. Architectural Ablation Analysis

Architecture Variant	Step Accuracy	Latency (P95)	Cost / Query	Key Observations
Baseline: Full LLM Deeplink Mapping				
Variant A: Hybrid BM25 + Dense Embedding Retrieval				
Variant B: Pure Rules-Based Deeplink Mapping				

21988027121789119435
Binny Lakra|binny.l|Samsung Electronics|CEO|Device eXperience|Device eXperience CTO|Samsung Resea

## 6. Known Edge Cases & System Limitations

* Document any unhandled multi-intent edge cases, domain gaps, or settings hierarchy variations observed durin

21988027121789119435
Binny Lakra|binny.l|Samsung Electronics|CEO|Device eXperience|Device eXperience CTO|Samsung Research|SRI-Bangalore|Tech Strategy